

35.10. BONHOMME verrouillé : on s'en sert, on ne le modifie pas

Une marque sous le dessin, /* BONHOMME : verrouillé */ : le jeu s'en sert comme avant, l'atelier refuse de le retoucher.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : une ligne de plus dans `perso_BONHOMME.cpp`, juste sous le dessin : `/* BONHOMME : verrouillé */`. C'est un **commentaire** : le compilateur l'ignore, le jeu est **exactement le même**.

C'est l'atelier qui la lit. Tant qu'elle est là, BONHOMME est un **modèle** : on le voit, on l'appelle partout (`montrerHeros`, deux exemplaires, qui marchent), mais **on ne le modifie plus** — le pinceau, le miroir, l'effacer et le renommer sont refusés, et une frappe dans ses rangées est défaite aussitôt. Il ne se supprime pas non plus.

Pour la poser ou l'ôter, pas besoin de l'écrire : « 🔒 Verrouiller » / « 🔓 Déverrouiller » dans « 🧩 Les tuiles », ou sur BONHOMME dans « 🏠 Tout le jeu ». Pour une version différente : « 📄 Créer une variante », une copie qu'on peut changer.

Le bilan du cours : `perso_BONHOMME.cpp` — à quoi il ressemble, verrouillé ; `heros.cpp` — ce qu'on en fait ; `principal.cpp` — quand et où. Trois fichiers, trois rôles, et un modèle qu'on ne casse pas par mégarde.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)
- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de 16 × 16](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **504 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          50 o ← la console, à l'allumage — la mise en route de la console
montrerHeros()     219 o ← main() — ta fonction
main()             52 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
RangerLutins       11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles       31 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles            64 o ← CopierTuiles — 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert   10 o ← InstallerTransfert — la petite routine qui copie les lutins
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

```
sprite()      x 4   218 o sur place, dans montrerHeros()
image()       x 1    6 o sur place, dans main() + AttendreImage (28 o)
sprite16()    x 1    0 o sur place, dans montrerHeros()
Tes fonctions appelées :
montrerHeros() x 2   29 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le même jeu : la seule différence est la marque, dans perso_BONHOMME.cpp.

#include "personnages.cpp" // la liste : BONHOMME
#include "heros.cpp"       // montrerHeros(numero, x, y)
```

2. UNE VARIABLE — X

```
uint8_t x = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    montrerHeros(4, 104, 64);

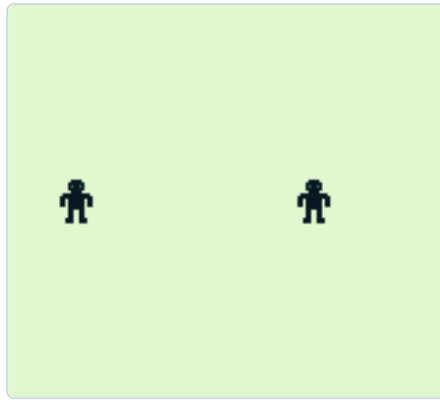
    while (true) {
        image();
        if (x < 120) x = x + 1;
        montrerHeros(0, x, 64); // verrouillé, il s'appelle comme avant
    }
}
```

3. Ce qui se passe, étape par étape

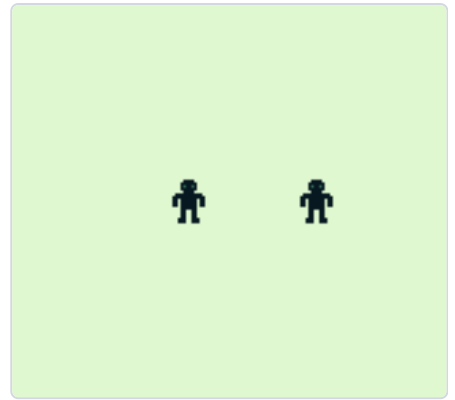
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



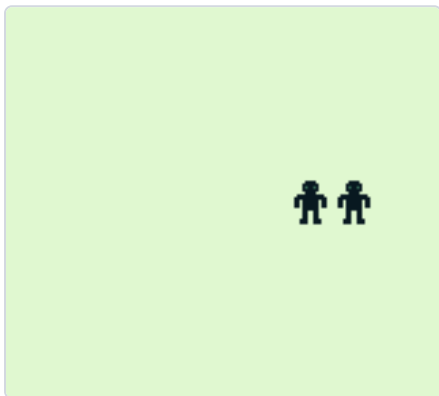
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Exactement l'étape d'avant : le bonhomme marche, l'autre attend. Le verrou ne change rien au jeu.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ verrouillé, BONHOMME marche comme avant

✓ et le second est toujours là

Ensuite : **35.11. La fonction `texteTitre()` — un mot en GROSSES lettres de titre** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++